개인 회고

진행하면서 막혔던 부분과 해결

막힌 부분	원인	해결
원본 노트북에 설치 셀이 없음	원본은 강사 서버의 conda 환경에서 실행해서 패키지가 이미 깔려 있었다. 저장소 코드를 읽어 보니 `pyrallis`와 OpenAI `clip`이 필요한데 코랩에 없다	첫 셀에 설치 줄을 넣고, 바로 위에 원본에 없던 줄이라는 표시와 이유를 주석으로 남겼다
import 셀에서 `CLIPFeatureExtractor` 에러	코랩 기본 `transformers`(5.18)에서 이 이름이 빠졌다. `transformers`만 낮추면 이번엔 `diffusers 0.40`과 `huggingface-hub` 버전이 충돌했다	코랩 터미널에서 조합을 시험해 `transformers<5` + `diffusers==0.30.3`으로 맞췄다
`%load_ext autoreload` 에러	코랩의 Python 3.13에는 이 확장이 쓰는 `imp` 모듈이 없다	코드를 고칠 때 다시 불러오는 개발용 기능이라 결과와 무관해서 주석 처리했다
모델 주소 `runwayml/stable-diffusion-v1-5`를 쓸 수 없음	runwayml이 Hugging Face에서 저장소를 내렸다	같은 가중치를 옮겨 둔 공식 미러 `stable-diffusion-v1-5/stable-diffusion-v1-5`로 바꿨다
정상 이미지를 불러오는 셀에 그림이 안 나옴	코랩에서는 PIL의 `show()`가 셀 아래에 그림을 띄우지 않는다	`display()`로 바꿨다
PCB 셀이 읽는 `normal.png`가 저장소에 없음	강사가 따로 넣어 둔 파일로 보인다	원본 노트북 출력에 남은 PCB 사진을 꺼내 `normal.png`로 저장하고 코랩에 올렸다

원본 코드는 그대로 두고, 코랩에서 안 되는 줄만 바로 위에 원본 줄과 바꾼 이유를 주석으로 남겼다.

실습을 통해 알게 된 것

공장에서는 불량이 드물어서 결함 사진을 모으기 어렵다. 이번 실습의 AnomalyAny는 정상 사진 1장과 결함을 설명하는 문장만으로, 추가 학습 없이 Stable Diffusion에서 결함이 있는 사진을 만든다. 정상 사진에 노이즈를 조금만 섞어서 출발하기 때문에(200스텝 중 마지막 60스텝만 진행) 물체의 모양과 배치는 거의 그대로 남고, 결함 단어(`faded`, `crack`)가 그림에 강하게 반영되도록 주의(attention) 맵을 키우면서 결함을 그려 넣는다.

단어별 주의 맵을 그려 보니 물체 단어(`table`)는 물체 전체를, 결함 단어(`faded`)는 상판 가운데 좁은 곳을 비췄고, 실제로 그 자리에 얼룩이 생겼다. 결함 단어의 주의 맵이 결함 위치를 알려 주므로, 결함 이미

지와 함께 "어디가 불량인지"를 나타내는 마스크도 얻을 수 있겠다고 생각했다. 이상 탐지 모델을 평가하려면 이런 픽셀 단위 정답이 필요하다.

하지만 만든다고 다 쓸 수 있는 것은 아니었다. 헤이즐넛 균열은 4장 중 2장만 그럴듯했고, 나머지는 배경에 엉뚱한 자국이 생기거나 정상과 거의 같았다. 탁자는 "햇빛에 바램"보다 얼룩에 가까웠고, PCB는 균열 대신 커넥터가 망가지거나 글자와 배선이 뭉개졌다. 생성한 결함 데이터는 사람이 보고 골라야 하고, 물체에 맞는 결함 문장을 고르는 것이 중요하다는 것을 알게 됐다.

개인 실험 요약

원본 코드는 건드리지 않았고, 실험 셀에는 `[개인 실험]` 표시를 했습니다.

1. **PCB에 맞는 결함 문장**: 원본의 `crack` 문장으로는 PCB에 균열이 생기지 않고 커넥터가 망가지거나 글자·배선이 뭉개지기만 해서, PCB에 실제로 생기는 결함을 문장으로 주면 나아지는지 궁금했습니다. "부품 누락(missing component)"과 "탄 자국(burnt mark)" 두 문장을 시드 0, 1로 2장씩 만들었습니다. 탄 자국 시드 1은 IC 주변이 까맣게 탄 듯한 자국으로 덮였고, 결함 단어의 주의가 몰린 자리와 같았습니다. 부품 누락 시드 0은 부품이 있던 자리가 빈 패드로 바뀌어 부품이 빠진 것처럼 보였습니다. 나머지 2장은 결함이 뚜렷하지 않아서, 4장 중 2장이 결함처럼 보였습니다(`crack`은 원본 셀 4장 중 0장). 원본 셀은 시드가 출력되지 않아 같은 시드로 비교하지 못했고 장수도 적어서 단정할 수는 없지만, 물체에 맞는 결함 문장을 고르는 것이 중요하다고 느꼈습니다. 또 같은 시드끼리는 문장이 달라도 기판 모양이 거의 같아서, 전체 모습은 시드가 정하고 결함이 생기는 자리는 결함 단어의 주의가 정한다는 것도 알게 됐습니다.

어려웠던 것

라이브러리 버전 맞추기가 가장 어려웠다. 코랩에 깔린 최신 버전으로는 import부터 막혔고, 하나를 낮추면 다른 라이브러리와 충돌했다. 터미널에서 조합을 하나씩 시험한 끝에 `transformers 4.57.6` + `diffusers 0.30.3`으로 돌아가는 것을 확인했다. 연구 코드는 만들 때의 버전을 정확히 고정해 두지 않으면 시간이 조금만 지나도 그대로 돌지 않는다는 것을 느꼈다.

좋았던 점

코랩 Pro의 A100에서 돌려서 결함 이미지 1장이 3분 남짓(개인 실험 셀 출력 기준 187₩~197초)으로 빨리 만들어졌다.

현장에 쓴다면

생성한 이미지를 바로 불량 데이터로 쓰지 않고, 사람이 보고 실제 결함처럼 보이는 것만 고를 것 같다. 또 결함이 엉뚱한 곳(배경)에 생기지 않도록 마스크로 결함 위치를 지정하고, 결함 문장은 "균열" 같은 일반적인 말보다 그 제품에 실제로 생기는 결함으로 정할 것이다. 시드를 고정해서 기록해 두면, 좋은 결과가 나왔을 때 같은 이미지를 다시 만들 수 있다.

앞으로 해 볼 것

이번에는 PCB 결함 문장을 2장씩만 시험해서, 시드를 더 늘려 결함 문장마다 몇 장 중 몇 장이 쓸 만한지 비율로 비교해 보고 싶다. 헤이즐넛에서 결함이 배경에 생긴 문제는 저장소에 들어 있는 전경 마스크(`fg_mask`)로 결함 위치를 물체 안으로 제한하면 줄어드는지 확인해 보고 싶다. 마지막으로, 만든 결함 이미지와 결함 단어의 주의 맵으로 만든 마스크를 6주차 이상 탐지 모델(PaDiM)의 테스트 데이터로 넣어서, 진짜 결함 사진처럼 잘 잡히는지 확인해 보고 싶다.